

Kubernetes

Containers Overview

Complete Notes + Interview Questions & Answers

1. What is a Container?

A container is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries, and settings.

Think of it as a self-contained lunchbox — it carries the food, utensils, and napkin together. No matter which kitchen (server) you open it in, everything you need is already inside.

Formal Definition:

Containers are isolated user-space instances of an operating system, sharing the host OS kernel but packaging application code and its dependencies together.

1.1 The Problem Containers Solve

Before containers, developers faced the classic "It works on my machine" problem:

- Dev machine runs Python 3.9, production runs Python 3.6 → app breaks
- Library versions conflict between two apps on the same server
- Setting up a new environment takes hours or days
- No reliable way to replicate the exact same environment

Containers solve this by packaging the app AND its entire environment together.

1.2 Containers vs Virtual Machines

Feature	Virtual Machine (VM)	Container
OS	Full OS per VM (GBs)	Shares host OS kernel
Size	Gigabytes	Megabytes
Boot Time	Minutes (30-60s)	Milliseconds to seconds
Isolation	Hardware-level	OS process-level
Portability	Heavy, less portable	Highly portable
Resource Usage	High overhead	Minimal overhead
Use Case	Full isolation needed	Microservices, scaling

Key Insight:

VMs virtualize hardware. Containers virtualize the OS. That is why containers are so much faster and lighter.

2. Container Architecture

2.1 What is Inside a Container?

- Application code (your Node.js / Python / Java app)
- Runtime (Node.js engine, Python interpreter, JVM)
- Libraries and dependencies (npm packages, pip packages)
- Configuration files (env variables, config.json)
- A minimal filesystem slice (not a full OS)

2.2 Linux Kernel Features That Power Containers

Containers are not magic — they use two core Linux kernel features:

Namespaces (Isolation)

Namespaces give each container its own isolated view of the system:

- pid namespace — container has its own process tree (PID 1 inside container)
- net namespace — container has its own network interface, IP address
- mnt namespace — container has its own filesystem mount points
- uts namespace — container has its own hostname
- ipc namespace — isolated inter-process communication
- user namespace — container can have its own user/group IDs

cgroups (Resource Control)

Control Groups (cgroups) limit and monitor resource usage per container:

- CPU limits — "this container gets max 0.5 CPU cores"
- Memory limits — "this container cannot use more than 512MB RAM"
- Disk I/O limits — throttle read/write speeds
- Network bandwidth limits

2.3 Container Layers (Union File System)

Container images are built in layers using a Union File System (like OverlayFS):

```
Layer 4: Your App Code           (read-write, per container)
Layer 3: pip install flask       (read-only)
Layer 2: apt install python3     (read-only)
Layer 1: Ubuntu 22.04 base image (read-only)
```

Why this matters:

- Layers are cached — if Layer 2 didn't change, Docker reuses it (fast builds)
- Multiple containers can share the same read-only layers (saves disk space)
- Only the top write layer is unique per container

3. Container Lifecycle

3.1 The Journey: Dockerfile → Image → Container

Step 1 — Dockerfile (Blueprint):

A text file with instructions on how to build the image.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "app.py"]
```

Step 2 — Image (Snapshot):

Build the Dockerfile into an image. This is a read-only, immutable snapshot.

```
docker build -t my-app:v1 .
```

Step 3 — Container (Running Instance):

Run the image to create a live container.

```
docker run -d -p 8080:8080 my-app:v1
```

Step 4 — Registry (Distribution):

Push the image to a registry so others / Kubernetes can pull it.

```
docker push myrepo/my-app:v1
```

3.2 Container States

- Created — container created but not started
- Running — container process is actively executing
- Paused — process is suspended (SIGSTOP)
- Stopped/Exited — process finished or was stopped
- Dead — container failed during removal

3.3 Key Docker Commands

Command	What it Does	Example
docker build	Create image from Dockerfile	docker build -t app:v1 .
docker run	Start a container from image	docker run -d app:v1
docker ps	List running containers	docker ps -a (all)
docker stop	Gracefully stop container	docker stop <id>
docker rm	Remove stopped container	docker rm <id>
docker logs	View container output	docker logs <id>
docker exec	Run command inside container	docker exec -it <id> bash
docker pull	Download image from registry	docker pull nginx:latest
docker push	Upload image to registry	docker push repo/app:v1

4. Container Properties (The 4 Superpowers)

Superpower 1: Isolation

Each container runs in its own isolated environment:

- Own filesystem — container A cannot see container B's files
- Own network namespace — different IP, different ports

- Own process space — PID 1 inside the container is independent
- If one container crashes, others are unaffected

Real-world example: App A needs OpenSSL 1.1, App B needs OpenSSL 3.0. Both run on the same server in their own containers with zero conflict.

Superpower 2: Portability

"Build once, run anywhere" — the core promise of containers.

- Developer's laptop → Staging server → Production cloud — same image, same behavior
- Works on AWS, GCP, Azure, on-premise, all using the same container image
- Eliminates environment-specific bugs

This is why Kubernetes can schedule a container on any node in the cluster — portability makes it possible.

Superpower 3: Speed

- Start time: milliseconds to ~1 second (vs 30-60s for VMs)
- Kubernetes uses this to auto-scale: traffic spikes → new containers up in seconds
- CI/CD pipelines run tests in fresh containers in seconds
- Fast rollbacks — just switch to the previous image tag

Superpower 4: Immutability

Container images are immutable (read-only). This is a design principle:

- You never modify a running container to deploy changes
- You build a NEW image for every version
- Old image tag still exists — rollback is trivial
- "Cattle, not pets" — containers are disposable and replaceable

This is foundational to GitOps and Kubernetes deployment strategies (Rolling Update, Blue-Green, Canary).

5. Containers in Kubernetes

5.1 How Kubernetes Uses Containers

Kubernetes does NOT run containers directly. The architecture is:

```
Kubernetes API Server
|
kubelet (on each Node)
|
Container Runtime (containerd / CRI-O)
|
Containers (inside Pods)
```

Kubernetes manages containers via the Container Runtime Interface (CRI). Docker was originally used but is now replaced by containerd as the default runtime in most clusters.

5.2 Pod: The Smallest Unit in Kubernetes

In Kubernetes, containers always run inside a Pod. A Pod is a wrapper around one or more containers:

- Containers in the same Pod share the same network namespace (same IP address)
- They share the same storage volumes
- They are scheduled together on the same node
- They start and stop together

Typical pattern: one main container + optional sidecar containers (logging agent, proxy, etc.) in one Pod.

5.3 Container Runtime Evolution

Runtime	Status	Notes
Docker	Deprecated in K8s v1.24+	Still used for building images
containerd	Current default	Lightweight, production-grade
CRI-O	Alternative	Designed specifically for Kubernetes
gVisor	Specialized	Extra isolation via sandbox kernel

5.4 Container Resource Management in K8s

Kubernetes lets you define resource requests and limits per container:

```
resources:
  requests:           # minimum guaranteed
    memory: "128Mi"
```

```
cpu: "250m"           # 250 millicores = 0.25 CPU
limits:                # maximum allowed
memory: "512Mi"
cpu: "500m"
```

- requests — used by scheduler to decide which node to place the Pod
- limits — enforced by cgroups; container killed if it exceeds memory limit
- 250m CPU = 0.25 cores (1000m = 1 full core)

6. Container Best Practices

6.1 Image Best Practices

- Use minimal base images (alpine, distroless) — smaller = faster pull, smaller attack surface
- One process per container — do not run multiple services in one container
- Never store secrets in images — use Kubernetes Secrets or Vault
- Use specific image tags (app:v1.2.3), never latest in production
- Leverage layer caching — put rarely-changing layers first in Dockerfile
- Run as non-root user — add USER 1001 in Dockerfile for security
- Use .dockerignore to exclude node_modules, .git, etc.

6.2 Runtime Best Practices

- Set resource requests and limits for every container
- Configure liveness and readiness probes
- Use read-only filesystems where possible (readOnlyRootFilesystem: true)
- Avoid privileged containers in production
- Implement graceful shutdown handling (SIGTERM signal)

7. Interview Questions & Answers

These are real questions asked in DevOps, SRE, and Cloud Engineer interviews. Study the structure of each answer — not just the content.

Q1. What is a container and how is it different from a virtual machine?

How to Answer:

Start with the definition, then explain the key difference, then give the practical impact.

Sample Answer:

A container is a lightweight, isolated runtime environment that packages an application with all its dependencies — code, libraries, runtime — but shares the host operating system's kernel. A VM, in contrast, includes a full guest OS on top of a hypervisor.

The key difference is resource efficiency: a container starts in milliseconds and is megabytes in size, while a VM takes minutes to boot and consumes gigabytes. This is because containers use Linux namespaces for isolation and cgroups for resource limiting — no need for a full OS per instance.

In practice, this means Kubernetes can spin up 50 container replicas in seconds during a traffic spike — something impossible with VMs.

☆ *Interview Tip: Always end with a real-world impact statement. Interviewers want to know you understand WHY it matters, not just WHAT it is.*

Q2. What are Linux namespaces and cgroups? How do they relate to containers?**How to Answer:**

These are the two Linux kernel primitives that make containers possible. Know both.

Sample Answer:

Containers are fundamentally built on two Linux kernel features:

Namespaces provide isolation. Each container gets its own view of the system — its own process tree (pid namespace), network stack (net namespace), filesystem mounts (mnt namespace), and hostname (uts namespace). From inside the container, it looks like a separate machine.

cgroups (Control Groups) provide resource control. They allow the kernel to limit, account for, and isolate resource usage — CPU, memory, disk I/O — per container. If a container hits its memory limit, the kernel kills its process (OOMKilled), not the entire host.

Together: namespaces give isolation, cgroups give resource guarantees. Docker and containerd are user-space tools that automate setting up these kernel features for each container.

☆ *Interview Tip: If asked a follow-up about OOMKilled in Kubernetes — it means the container exceeded its memory limit (cgroup enforcement). Fix: increase memory limit or fix the memory leak.*

Q3. Explain the difference between a Docker Image and a Docker Container.

How to Answer:

Use the class vs instance analogy — it is clean and every engineer relates to it.

Sample Answer:

An image is a read-only, immutable blueprint — like a class in object-oriented programming. It contains the application code, dependencies, and configuration, built in layers.

A container is a running instance of that image — like an object instantiated from a class. It has the image's read-only layers plus a thin, writable layer on top for runtime state.

You can run multiple containers from the same image simultaneously. Each container is independent — stopping one does not affect others. When a container is deleted, its writable layer is gone, but the image remains unchanged.

In Kubernetes terms: the image is stored in a registry (DockerHub, ECR, GCR), and each Pod creates a container from that image on the node.

☆ *Interview Tip: Interviewers sometimes ask: 'What happens to data when a container restarts?' Answer: it is lost (ephemeral), unless mounted to a volume. Persistent data needs PersistentVolumes in Kubernetes.*

Q4. What is container image layering and why does it matter?

How to Answer:

Explain the concept, then explain the two main benefits: caching and storage efficiency.

Sample Answer:

Container images are built in layers using a Union File System (commonly OverlayFS in Linux). Each instruction in the Dockerfile creates a new read-only layer on top of the previous one.

Why it matters — two key benefits:

1. Build caching: Docker caches each layer. If you change only your application code (top layer), Docker reuses all the cached layers below it (OS, runtime, dependencies). A build that would take 5 minutes becomes 10 seconds.

2. Storage efficiency: Multiple containers running the same image share the same read-only layers on disk. Only the thin writable layer per container is unique. This saves significant disk space in production clusters.

Best practice tip: Always put the most frequently changing instructions (COPY your code) at the bottom of the Dockerfile, and stable instructions (RUN apt install, RUN pip install) near the top so they get cached.

☆ *Interview Tip: Interviewers love follow-ups like 'How would you optimize a slow Docker build?' Layer ordering and .dockerignore are your answers.*

Q5. What is the Container Runtime Interface (CRI) in Kubernetes? Why was Docker deprecated?

How to Answer:

This shows you understand K8s internals beyond surface level.

Sample Answer:

The CRI is a plugin interface that allows Kubernetes (specifically kubelet) to use different container runtimes without being tightly coupled to any one implementation. It defines a standard gRPC API for starting, stopping, and managing containers.

Docker was deprecated as a Kubernetes runtime in version 1.20 and removed in v1.24. The reason: Docker is not CRI-compatible natively. Kubernetes was using a shim called dockershim to translate CRI calls to Docker's API — this added complexity and maintenance overhead for the Kubernetes team.

The replacement is containerd, which IS CRI-compatible natively. Containerd is actually the core runtime that Docker itself uses underneath — so in practice, the containers running in your cluster look and behave exactly the same. Only the management layer changed.

Other CRI runtimes: CRI-O (designed specifically for Kubernetes), gVisor (sandboxed runtime for extra isolation).

☆ *Interview Tip: Important clarification: Docker is still the standard tool for BUILDING images (docker build). It is only deprecated as the Kubernetes runtime. Your docker images still work perfectly in K8s.*

Q6. What are resource requests and limits in Kubernetes containers?

How to Answer:

Explain both concepts, their purpose, and what happens when limits are exceeded.

Sample Answer:

In Kubernetes, each container can declare two resource settings:

Requests are the minimum resources the container needs. The Kubernetes scheduler uses requests to decide which node has enough capacity to run the Pod. If no node has enough, the Pod stays Pending.

Limits are the maximum resources the container is allowed to use, enforced by cgroups:

- CPU limit: if exceeded, the container is throttled (slowed down), not killed
- Memory limit: if exceeded, the container is killed with OOMKilled (Out of Memory)

Best practice: always set both. Containers without limits can consume all resources on a node, starving other workloads (the 'noisy neighbor' problem). QoS classes in Kubernetes are determined by whether requests and limits are set and equal.

```
resources:
  requests:
    cpu: "100m"      # 0.1 core guaranteed
    memory: "128Mi"
  limits:
    cpu: "500m"      # throttled if exceeded
    memory: "256Mi" # killed (OOMKilled) if exceeded
```

☆ Interview Tip: If asked about QoS classes: *Guaranteed* (requests == limits), *Burstable* (requests < limits), *BestEffort* (no requests/limits). *Guaranteed pods are evicted last during node pressure.*

Q7. What is the difference between CMD and ENTRYPOINT in a Dockerfile?

How to Answer:

This is a common trap question. Be precise about how they interact.

Sample Answer:

Both define what runs when a container starts, but they interact differently:

ENTRYPOINT defines the main executable that always runs. It cannot be overridden by docker run arguments (only by --entrypoint flag).

CMD provides default arguments to ENTRYPOINT, or if ENTRYPOINT is not set, CMD is the command itself. CMD CAN be overridden by passing arguments to docker run.

Example:

```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

Running `docker run myapp` runs: `python app.py`

Running `docker run myapp test.py` runs: `python test.py` (CMD overridden)

Best practice: use ENTRYPOINT for the fixed executable, CMD for default arguments that users might want to override.

☆ Interview Tip: In Kubernetes, the Dockerfile ENTRYPOINT maps to 'command' in the Pod spec, and CMD maps to 'args'. Kubernetes 'command' overrides Dockerfile ENTRYPOINT if set.

Q8. How do you debug a container that is crashing in Kubernetes (CrashLoopBackOff)?

How to Answer:

This is a practical troubleshooting question. Show a systematic approach.

Sample Answer:

I follow a systematic process to debug CrashLoopBackOff:

Step 1 — Check Pod events:

```
kubectl describe pod <pod-name>
```

This shows the exit code (e.g., exit code 1 = app error, 137 = OOMKilled, 139 = segfault), restart count, and any scheduling or image pull errors.

Step 2 — Check logs (previous run):

```
kubectl logs <pod-name> --previous
```

The `--previous` flag shows logs from the last crashed instance, which is key since the current container may not have any logs yet.

Step 3 — Check resource limits:

If exit code is 137 and the reason is OOMKilled, the container exceeded its memory limit.

Solution: increase the memory limit or profile the application for memory leaks.

Step 4 — Shell into the container (if it stays up briefly):

```
kubectl exec -it <pod-name> -- /bin/sh
```

Step 5 — Override the command temporarily:

Set command: `["sleep", "3600"]` in the Pod spec to keep it alive and investigate manually.

☆ Interview Tip: Common causes of CrashLoopBackOff: app crash on startup (bad config, missing env var), OOMKilled (memory limit too low), image pull error (wrong tag, no registry access), liveness probe too aggressive.

Q9. What is container image immutability and why is it important in production?

How to Answer:

Connect immutability to real DevOps practices: rollbacks, security, auditability.

Sample Answer:

Immutability means that once a container image is built and tagged, it is never changed. If you need to update the application, you build a new image with a new tag. The old image remains intact.

Why this matters in production:

Reliable rollbacks: if v2 is broken, you deploy v1 — it is the exact same image that worked before, byte for byte. No surprises.

Auditability: every deployed version is traceable. You can look at the image tag in Git and know exactly what code, libraries, and configuration were running.

Security: you can scan images before deployment. An immutable image that passes a security scan will not change between scan time and deploy time.

Consistency: what runs in staging is what runs in production. No one SSHed into the container and made changes.

The anti-pattern to avoid: using the 'latest' tag in production. Latest is mutable — it points to a different image every time you build. This breaks repeatability and rollbacks.

☆ *Interview Tip: Connect this to GitOps: in GitOps, the Git repository is the source of truth. Every deployment is a specific, immutable image tag committed to Git. Kubernetes then reconciles the running state to match.*

Q10. What security best practices do you follow for containers in production?

How to Answer:

Show depth across image security, runtime security, and Kubernetes security contexts.

Sample Answer:

Container security operates at multiple layers:

Image-level security:

- Use minimal base images (distroless or alpine) to reduce the attack surface — fewer packages = fewer vulnerabilities
- Scan images with tools like Trivy, Snyk, or AWS ECR scanning before deployment
- Run as a non-root user (add USER 1001 in Dockerfile)
- Never bake secrets into images — use Kubernetes Secrets or Vault
- Pin specific image tags, never use latest in production

Runtime security (Kubernetes SecurityContext):

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
```

Cluster-level security:

- Use Pod Security Admission (PSA) to enforce security standards cluster-wide
- Implement Network Policies to restrict container-to-container traffic
- Use OPA/Gatekeeper to enforce image registry policies (only allow images from approved registries)

☆ Interview Tip: Mention the principle of least privilege — containers should have exactly the permissions they need, nothing more. This limits blast radius if a container is compromised.

8. Quick Reference Cheat Sheet

Key Terms

Term	Definition	K8s Relevance
Container	Isolated process with all dependencies	Runs inside a Pod
Image	Read-only blueprint (layers)	Pulled from registry to node
Dockerfile	Recipe to build an image	CI/CD builds image from this
Registry	Image storage (DockerHub, ECR)	Kubelet pulls from registry
containerd	Container runtime	Default K8s runtime since v1.24

Namespace	Kernel isolation mechanism	Gives container its own network/PID
cgroups	Kernel resource limiter	Enforces K8s resource limits
OOMKilled	Memory limit exceeded	Increase memory limit or fix leak
CrashLoopBackOff	Container keeps crashing	Check logs --previous, describe pod

The 4 Container Properties to Remember

- Isolation — own filesystem, network, processes via namespaces
- Portability — same image runs anywhere (laptop, AWS, GCP)
- Speed — millisecond startup enables Kubernetes auto-scaling
- Immutability — new version = new image tag, old tag = reliable rollback